Proyecto Final – Big Data y Machine Learning con Spark ML

Curso de Big Data

1. Propósito del proyecto

El proyecto final tiene como objetivo resolver un problema real de **detección, clasificación, predicción, regresión o segmentación** utilizando **Spark ML** como capa principal de Machine Learning.

El foco estará en diseñar y justificar una solución completa: desde la obtención y preparación de los datos hasta el entrenamiento, evaluación, interpretación y análisis de escalabilidad del modelo.

Cada grupo deberá construir una arquitectura donde los datos sean obtenidos mediante **una única vía principal de alimentación**, escogida y justificada entre:

A. Spark: lectura y procesamiento directamente mediante Spark.

B. Dask: preparación inicial con Dask, persistencia del resultado y posterior lectura desde Spark.

C. Kafka: recepción de eventos mediante Kafka y procesamiento con Spark Structured Streaming.

Importante: Spark ML opera sobre DataFrames de Spark. Por tanto, Dask o Kafka pueden constituir la capa de ingesta o preparación, pero los datos deberán llegar finalmente a un DataFrame de Spark antes de entrenar o aplicar el modelo.

2. Continuidad con las tareas anteriores

El proyecto debe recuperar elementos desarrollados durante el curso y demostrar que el grupo comprende cómo se conectan las distintas capas de una arquitectura Big Data.

Cuando sea posible, se recomienda continuar con el mismo dominio o conjunto de datos utilizado en las tareas anteriores. Si el grupo cambia de dominio o dataset, deberá justificar la decisión.

Cada grupo deberá incluir una breve sección denominada **"Análisis de datos a problema de Machine Learning"**, identificando al menos cuatro elementos previamente trabajados y su transformación en el proyecto.

Ejemplos:

Elemento previo	**Transformación en el proyecto**
Pregunta analítica	Variable objetivo o decisión que se desea predecir
EDA y distribuciones	Selección y transformación de variables
Outliers o patrones detectados	Señales potenciales para detección o clasificación
Agregaciones o ventanas	Características de entrada para el modelo
Particionamiento de datos	Paralelismo y escalabilidad del entrenamiento
Streaming desde Kafka	Inferencia continua o procesamiento de eventos

3. Tipos de problemas permitidos

El problema debe tener una finalidad verificable. Se permiten, entre otros:

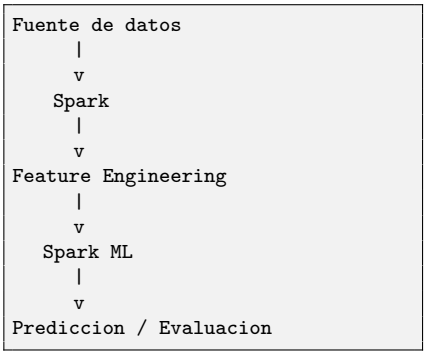
- clasificación binaria;
- clasificación multiclase;
- detección de eventos o condiciones;
- regresión o predicción de una variable numérica;
- predicción basada en variables temporales construidas mediante ventanas o rezagos;
- segmentación o clustering, siempre que exista una pregunta concreta que justifique la agrupación;
- detección de comportamientos atípicos mediante una estrategia justificable implementable con las herramientas utilizadas.

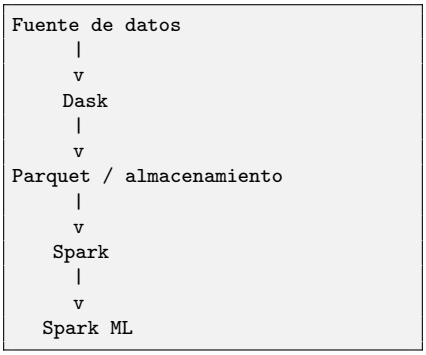
No se considerará suficiente ejecutar un modelo únicamente porque está disponible en Spark ML. El grupo deberá justificar por qué el problema requiere Machine Learning, qué representa la variable objetivo y cuál sería la utilidad de la predicción o detección obtenida.

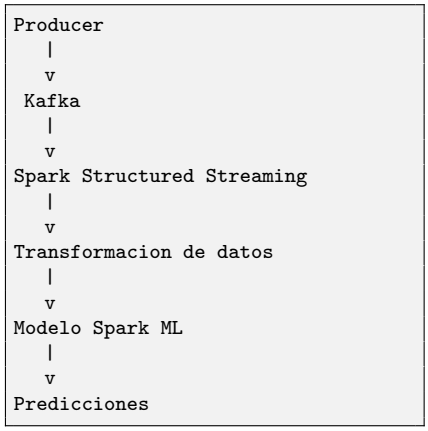
4. Arquitectura

El informe deberá incluir un diagrama de la arquitectura utilizada.

Ejemplos válidos:







Cada grupo deberá escoger **una** arquitectura principal y justificarla. No se evalúa positivamente incorporar Dask, Kafka y Spark simultáneamente si no existe una necesidad real.

5. Actividades

5.1. 1. Definición del problema

Definir claramente:

- el problema que se busca resolver;

- la unidad de análisis;
- la variable objetivo, si corresponde;
- qué significa una predicción correcta;
- qué decisión o conocimiento aporta el modelo;
- las limitaciones principales del problema.

La formulación debe estar expresada primero como un problema del dominio y posteriormente como un problema de Machine Learning.

5.2. 2. Ingesta y preparación

Implementar una única vía principal de alimentación: Spark, Dask o Kafka.

El grupo deberá describir:

- fuente y volumen de datos;
- esquema;
- variables utilizadas;
- tratamiento de nulos;
- duplicados;
- categorías;
- posibles valores anómalos;
- transformación necesaria antes del entrenamiento.

Si se utiliza Kafka, se deberá explicar además:

- qué representa cada evento;
- topic utilizado;
- key;
- número de particiones;
- relación entre la tasa de llegada y la capacidad de procesamiento.

5.3. 3. Ingeniería de características

Construir las variables requeridas por el modelo utilizando Spark.

Dependiendo del problema, pueden utilizarse:

- variables numéricas;
- codificación de variables categóricas;
- agregaciones;

- variables derivadas;
- características obtenidas mediante ventanas;
- rezagos temporales;
- conteos o tasas;
- escalamiento, cuando corresponda.

Se debe justificar cada transformación. No se evaluará positivamente incorporar variables sin explicar su relación con el problema.

5.4. 4. Pipeline de Spark ML

El procesamiento deberá organizarse mediante los componentes de `pyspark.ml`.

Ejemplo de constructor mínimo:

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=["x1", "x2", "x3"],
    outputCol="features"
)

pipeline = Pipeline(stages=[
    assembler,
    modelo
])
```

Este ejemplo entrega solamente la estructura. Cada grupo deberá seleccionar sus transformaciones, variables y modelo de acuerdo con su problema.

5.5. 5. Baseline obligatorio

Antes de evaluar modelos más complejos, el grupo deberá definir un **baseline**.

Ejemplos:

- predecir siempre la clase mayoritaria;
- utilizar una regla simple del dominio;
- utilizar la media o mediana en un problema de regresión;
- entrenar un modelo lineal o simple como referencia.

El objetivo es demostrar que el modelo propuesto mejora una alternativa razonable y no solamente que obtiene una determinada métrica.

5.6. 6. Modelos

Evaluar al menos **dos alternativas de modelado** disponibles en Spark ML cuando el tipo de problema lo permita.

Ejemplos:

- Logistic Regression;
- Decision Tree;
- Random Forest;
- Gradient-Boosted Trees;
- Linear Regression;
- Generalized Linear Regression;
- K-Means.

La selección debe estar justificada. No se requiere utilizar todos los algoritmos disponibles.

5.7. 7. Separación de entrenamiento y evaluación

Definir una estrategia de separación de datos consistente con el problema.

Se deberá evitar explícitamente el **data leakage**.

Si los datos poseen una componente temporal relevante, se espera una separación temporal en lugar de una división aleatoria cuando corresponda.

5.8. 8. Evaluación

Las métricas deben corresponder al problema.

Clasificación:

- precision;
- recall;
- F1-score;
- matriz de confusión;
- ROC-AUC y/o PR-AUC cuando corresponda.

No se deberá utilizar Accuracy como única métrica cuando exista desbalance de clases.

Regresión:

- MAE;
- RMSE;
- R^2;

- análisis de errores.

Clustering:

- Silhouette;
- caracterización de los grupos;
- interpretación de su utilidad.

Cada resultado debe seguir la lógica:

pregunta → experimento → resultado → interpretación

5.9. 9. Ajuste del modelo (OPCIONAL)

Realizar al menos una estrategia de ajuste o comparación de hiperparámetros utilizando herramientas de Spark ML, por ejemplo:

- TrainValidationSplit;
- CrossValidator;
- ParamGridBuilder.

El objetivo no es realizar una búsqueda exhaustiva, sino demostrar que el grupo entiende el efecto de los parámetros sobre el comportamiento del modelo.

5.10. 10. Escalabilidad

Realizar un experimento que permita observar el comportamiento del pipeline al modificar al menos una de las siguientes variables:

- volumen de datos;
- número de particiones;
- recursos disponibles;
- cantidad de variables;
- tasa de eventos, cuando se utilice Kafka.

Se deberá registrar el tiempo de ejecución y discutir si el cuello de botella se encuentra en:

- lectura;
- preparación;
- shuffle;
- feature engineering;
- entrenamiento;

- inferencia;
- almacenamiento;
- Kafka;
- número de particiones;
- capacidad del consumidor.

5.11. 11. Interpretación del modelo (OPCIONAL)

El grupo deberá explicar qué aprendió el modelo y qué variables parecen relevantes, utilizando las herramientas disponibles para el algoritmo seleccionado.

Cuando el algoritmo permita obtener importancia de variables, coeficientes u otra información interpretable, esta deberá ser analizada.

No basta con reportar una métrica final.

5.12. 12. Si se utiliza Kafka

Cuando la arquitectura seleccionada utilice Kafka, se deberá distinguir entre:

- **entrenamiento**: realizado sobre un DataFrame de Spark con un conjunto de datos definido;
- **inferencia**: aplicación del modelo entrenado sobre los eventos recibidos.

Una arquitectura esperada puede ser:

```
Datos historicos
      |
      v
Spark -> Spark ML -> modelo entrenado
                         |
                         v
Kafka -> Structured Streaming -> transformacion -> prediccion
```

El grupo deberá explicar cómo se administran los offsets, el checkpointing y qué ocurriría si el procesamiento se interrumpe temporalmente.

6. Preguntas mínimas que debe responder el proyecto

Al finalizar, el grupo deberá ser capaz de responder:

1. ¿Qué problema real intenta resolver?
2. ¿Por qué es un problema de Machine Learning?
3. ¿Cuál es la variable objetivo?
4. ¿Qué representa cada observación?
5. ¿Qué variables utiliza el modelo?
6. ¿Cómo se construyeron esas variables?

7. ¿Cuál es el baseline?
8. ¿Qué modelos se compararon?
9. ¿Qué métrica es la más importante y por qué?
10. ¿Existe desbalance, leakage o dependencia temporal?
11. ¿Cuál fue el principal cuello de botella?
12. ¿Cómo cambia el rendimiento al aumentar la escala?
13. ¿Qué limitaciones tiene la solución?
14. ¿Qué decisión podría tomarse utilizando el resultado del modelo?

7. Entregables

1. **Notebook ejecutable**, con todas las celdas ejecutadas y sus resultados visibles. Con problema, metodología, resultados, evaluación, escalabilidad y conclusiones.
2. **Diagrama de arquitectura**.
3. **Presentación y defensa**.
4. Código o archivos auxiliares necesarios para reproducir la solución.
5. Declaración de uso de herramientas de Inteligencia Artificial.

8. Criterios de evaluación

Criterio
Formulación y relevancia del problema
Diseño de arquitectura e ingesta
Preparación e ingeniería de características
Uso correcto de Spark ML y Pipeline
Baseline y comparación de modelos
Evaluación y selección de métricas
Análisis de escalabilidad y cuellos de botella
Interpretación y discusión de resultados
Reproducibilidad y calidad de la defensa

9. Restricciones

- El modelo principal debe implementarse utilizando **Spark ML**.
- No se acepta reemplazar Spark ML por scikit-learn como núcleo del proyecto.
- El proyecto debe utilizar una sola vía principal de ingesta: Spark, Dask o Kafka.
- La elección de la arquitectura debe estar justificada por el problema y no por acumular tecnologías.
- El notebook debe poder ejecutarse de principio a fin.

- Las conclusiones deben estar respaldadas por resultados medibles.
- Toda herramienta de IA utilizada debe ser declarada junto con la forma en que se verificó su resultado.

10. Foco de la defensa

La defensa no se centrará en recordar sintaxis. El grupo deberá justificar decisiones como:

- por qué escogió determinada arquitectura;
- por qué una variable puede o no utilizarse como feature;
- por qué una métrica es apropiada;
- cómo detectó un posible caso de leakage;
- por qué un modelo supera al baseline;
- qué ocurre al aumentar el volumen de datos;
- dónde se encuentra el cuello de botella;
- qué modificaría para llevar la solución a un entorno productivo.